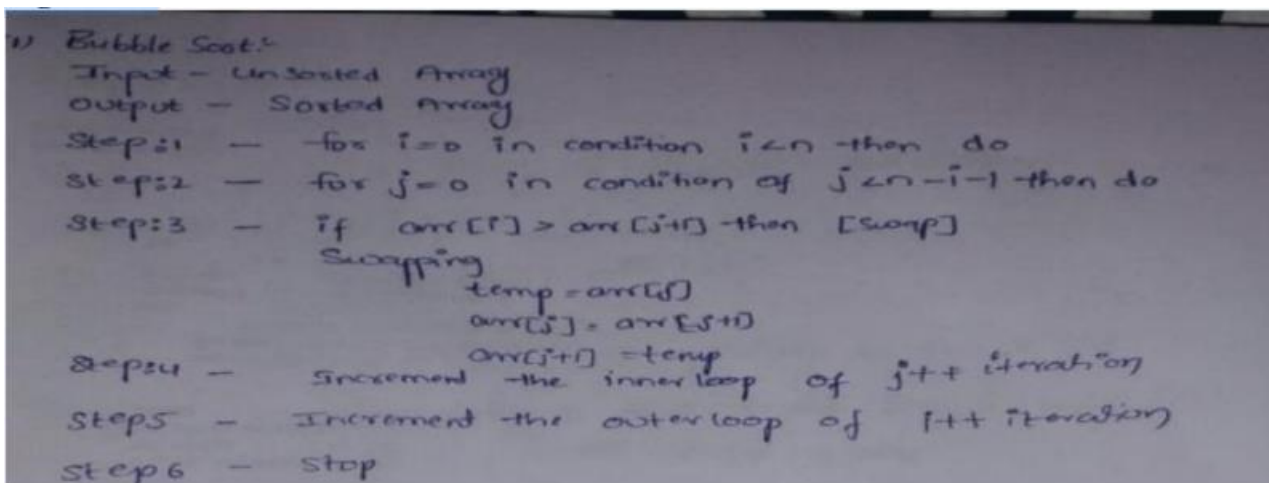


Practical: 1

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:



1) Bubble Sort:-
Input - Unsorted Array
Output - Sorted Array
Step:1 - for $i=0$ in condition $i < n$ -then do
Step:2 - for $j=0$ in condition of $j < n-i-1$ -then do
Step:3 - if $arr[i] > arr[j+1]$ -then [swap]
 Swapping
 temp = arr[i]
 arr[i] = arr[j+1]
 arr[j+1] = temp
Step:4 - Increment the inner loop of $j++$ iteration
Step:5 - Increment the outer loop of $i++$ iteration
Step:6 - stop

Program:

```
#include <iostream>

using namespace std;

class bubblesort
{
public:
    int size;
    int arr[100];
    int i;

    void arrayinput()
    {
```



```
cout << "enter the size of array" << endl;
```

```
cin >> size;
```

```
cout << "enter the array elements" << endl;
```

```
for (i = 0; i < size; i++)
```

```
{
```

```
    cin >> arr[i];
```

```
}
```

```
}
```

```
void printarray()
```

```
{
```

```
    for (i = 0; i < size; i++)
```

```
    {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
void bubblelogic()
```

```
{
```

```
    int i, j, temp;
```

```
    for (i = 0; i < size - 1; i++)
```

```
    {
```

```
        for (j = 0; j < size - i - 1; j++)
```

```
        {
```

```
            if (arr[j] > arr[j + 1])
```

```
        {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
};

int main()
{
    bubblesort bs;

    bs.arrayinput();

    cout << "before sorting" << endl;
    bs.printarray();

    cout << "after sorting" << endl;
    bs.bubblelogic();
    bs.printarray();

    return 0;
}
```

OUTPUT:

```
enter the size of array
5
enter the array elements
3
2
6
1
22
before sorting
3 2 6 1 22
after sorting
1 2 3 6 22

=== Code Execution Successful ===
```

Time complexity:

$\text{for } (i=0; i < n; i++) \{ \rightarrow O(n)$
 $\text{for } (j=0; j < n; j++) \{ \rightarrow O(n)$
 $\text{if } \text{arr}[j] > \text{arr}[j+1] : \rightarrow O(1)$
 $\text{temp} = \text{arr}[j]$
 $\text{arr}[j] = \text{arr}[j+1]$
 $\text{arr}[j+1] = \text{temp} \} \rightarrow O(1)$
 $\}$



The array is already sorted
Eg $\rightarrow [1, 2, 3, 4, 5]$

```
for (i=0; i < n-1; i++) {  $\rightarrow n-1$ 
    for (j=0; j < n-1; j++)  $\rightarrow n-1$ 
```

Number of comparisons = $n-1$
 $T(n) = n-1$

ignoring the constants and the lower-order term
 $T(n) = O(n)$

Average case - the input elements are in a random or partially unsorted order.

for Bubble sort approximately $1+2+3+\dots+(n-1)$

using $1+2+3+\dots+(n-1) = \frac{(n-1)n}{2}$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

ignoring constants
 $T(n) = O(n^2)$

Worst case The input arrangement requires the maximum amount of work

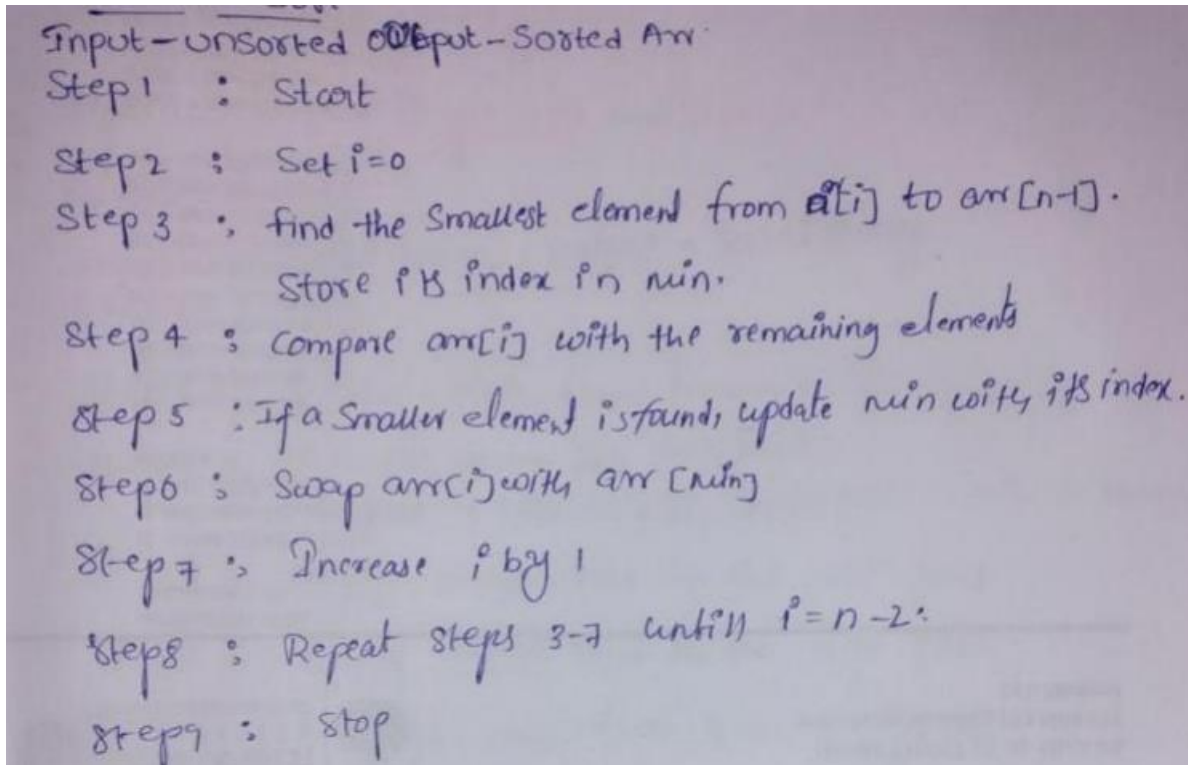
Number of comparisons
 $T(n) = (n-1) + (n-2) + \dots + 1$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = O(n^2)$$

SELECTION SORT:

ALGORITHM:



Input - unsorted array Output - Sorted Arr.

Step 1 : Start

Step 2 : Set $i=0$

Step 3 : find the smallest element from $arr[i]$ to $arr[n-1]$.
Store its index in min .

Step 4 : Compare $arr[i]$ with the remaining elements

Step 5 : If a smaller element is found, update min with its index.

Step 6 : Swap $arr[i]$ with $arr[min]$

Step 7 : Increase i by 1

Step 8 : Repeat steps 3-7 until $i = n-2$.

Step 9 : Stop

PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
class selectionsort
```

```
{
```

```
public:
```

```
    int size;
```

```
    int arr[100];
```

```
    int i;
```

```
    void arrayinput()
```

```
{
```

```
cout << "Enter the size of array" << endl;
```

```
cin >> size;
```

```
cout << "Enter the array elements" << endl;
```

```
for (i = 0; i < size; i++)
```

```
{
```

```
    cin >> arr[i];
```

```
}
```

```
}
```

```
void printarray()
```

```
{
```

```
    for (i = 0; i < size; i++)
```

```
    {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
void selectionlogic()
```

```
{
```

```
    int i, j, minIndex, temp;
```

```
    for (i = 0; i < size - 1; i++)
```

```
    {
```

```
        minIndex = i;
```

```
        for (j = i + 1; j < size; j++)
```

```
{  
    if (arr[j] < arr[minIndex])  
    {  
        minIndex = j;  
    }  
}  
  
temp = arr[i];  
arr[i] = arr[minIndex];  
arr[minIndex] = temp;  
}  
}  
};
```

```
int main()  
{  
    selectionsort ss;  
  
    ss.arrayinput();  
  
    cout << "Before sorting" << endl;  
    ss.printarray();  
  
    ss.selectionlogic();  
  
    cout << "After sorting" << endl;  
    ss.printarray();  
}
```



```
return 0;  
}
```

OUTPUT:

```
enter the size of array  
5  
enter the array elements  
2  
4  
1  
66  
22  
before sorting  
2 4 1 66 22  
after sorting  
1 2 4 22 66  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:

```
Void SelectionLogic ( ) {  
    int i, j, minIndex, temp;  —————> O(1)  
    for (i=0; i < size-1; i++) { —————> O(n)  
        minIndex = i;  —————> O(1)  
        for (j=i+1; j < size; j++) { —————> O(n)  
            if (arr[j] < arr[minIndex]) { —————> O(1)  
                minIndex = j;  —————> O(1)  
            }  
        }  
        temp = arr[i];  —————> O(1)  
        arr[i] = arr[minIndex]; —————> O(1)  
        arr[minIndex] = temp; —————> O(1)  
    }  
}
```



Best case Even if the array is already sorted, selection sort still searches for the minimum elements in remaining unsorted array.

So no. of comparisons does not decrease.

for pass 1 $\rightarrow n-1$, pass 2 $\rightarrow n-2$, last 1

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

ignoring constants

$$T(n) = O(n^2)$$

Average case

The average case occurs when the elements are in random order.

$$T(n) = (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2} \text{ — ignoring constants}$$

$$T(n) = O(n^2)$$

Worst case occurs when the array is arranged in an order that causes maximum rearrangement.

$$T(n) = (n-1) + (n-2) + \dots + 1$$

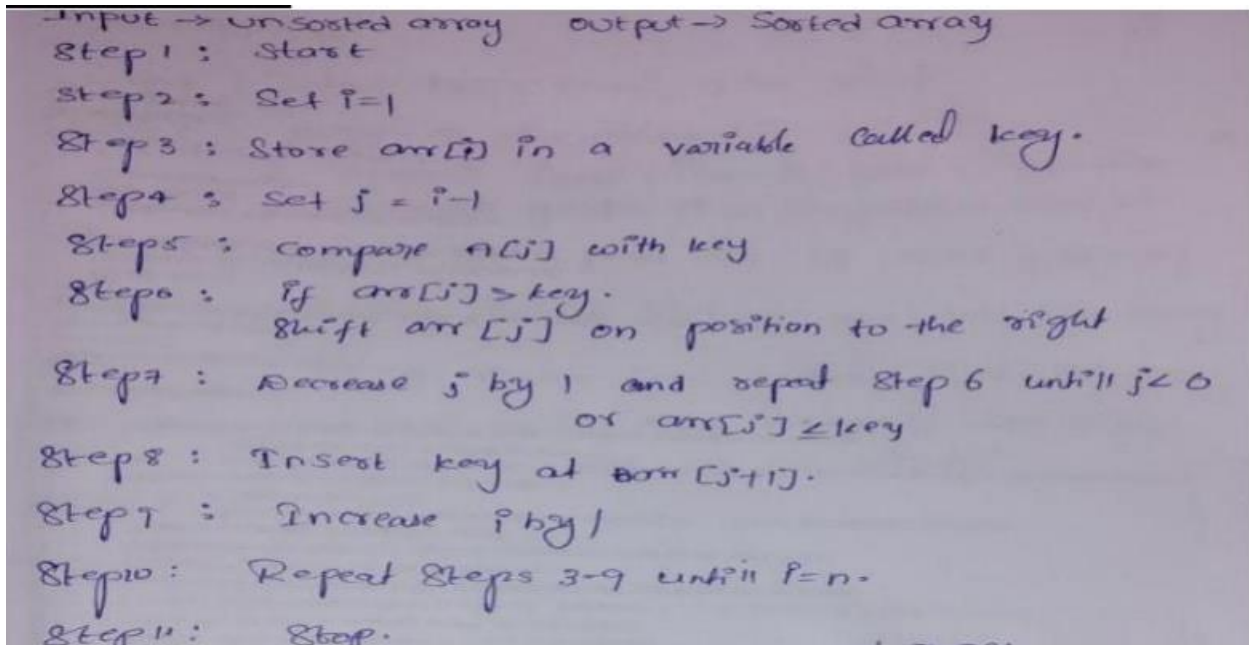
$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2} \text{ ignoring constants}$$

$$T(n) = O(n^2)$$

INSERCTION SORT:

ALGORITHM:



Input $\rightarrow$ Unsorted array output $\rightarrow$ Sorted array

Step 1 : Start

Step 2 : Set $i = 1$

Step 3 : Store $arr[i]$ in a variable called key.

Step 4 : Set $j = i - 1$

Step 5 : Compare $arr[j]$ with key

Step 6 : If $arr[j] > key$.
Shift $arr[j]$ on position to the right

Step 7 : Decrease j by 1 and repeat Step 6 until $j < 0$
or $arr[j] \leq key$

Step 8 : Insert key at $arr[j+1]$.

Step 9 : Increase i by 1

Step 10 : Repeat Steps 3-9 until $i = n$.

Step 11 : Stop.

PROGRAM:

```
#include <iostream>

using namespace std;

class insertionsort
{
public:
    int size;
    int arr[100];
    int i;

    void arrayinput()
    {
        cout << "Enter the size of array" << endl;
        cin >> size;
```

```
cout << "Enter the array elements" << endl;
for (i = 0; i < size; i++)
{
    cin >> arr[i];
}
}
```

```
void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
void insertionlogic()
{
    int i, j, key;

    for (i = 1; i < size; i++)
    {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
```

```
        arr[j + 1] = arr[j];  
        j = j - 1;  
    }
```

```
        arr[j + 1] = key;  
    }  
}  
};
```

```
int main()  
{  
    insertionSort is;  
  
    is.arrayInput();  
  
    cout << "Before sorting" << endl;  
    is.printArray();  
  
    is.insertionLogic();  
  
    cout << "After sorting" << endl;  
    is.printArray();  
  
    return 0;  
}
```

OUTPUT:



```
enter the size of array
5
enter the array elements
4
3
55
22
6
before sorting
4 3 55 22 6
after sorting
3 4 6 22 55

=== Code Execution Successful ===
```

TIME COMPLEXITY:

```
Void insertionSort () {
    int i, j, key;
    for (i=1; i<size; i++) {
        key = arr[i];
        j = i-1;
        while (j>=0 && arr[j]>key) {
            arr[j+1] = arr[j];
            j = j-1;
        }
        arr[j+1] = key;
    }
}
```



Best case - occurs when the array is already sorted

for every element, the condition:

$arr[j] > key$ is false immediately

therefore, the while loop does not perform any shifting
the for loop still runs: $(n-1)$ times

for each iteration

$key = arr[i] \rightarrow O(1)$

$j = i - 1 \rightarrow O(1)$

while condition $\rightarrow O(1)$

$arr[j+1] = key \rightarrow O(1)$

So

$$T(n) = (n-1) * O(1)$$

$$\boxed{T(n) = O(n)}$$

Average case occurs when elements are in random order

$i = 1 \rightarrow 1$

$i = 2 \rightarrow 2$

$i = 3 \rightarrow 3$

$i = n-1 \rightarrow n-1$

$$1 + 2 + 3 + \dots + (n-1)$$

$$\therefore T(n) = \frac{n(n-1)}{2} \quad \boxed{T(n) = O(n^2)}$$

Worst case occurs when the array is in reverse sorted order.

→ Every element has to move all the way to the beginning
for each iteration

$i = 1 \rightarrow 1$ shift

$i = 2 \rightarrow 2$ shift

...

$i = n-1 \rightarrow n-1$ shift.

$$T(n) = 1 + 2 + 3 + \dots + (n-1)$$

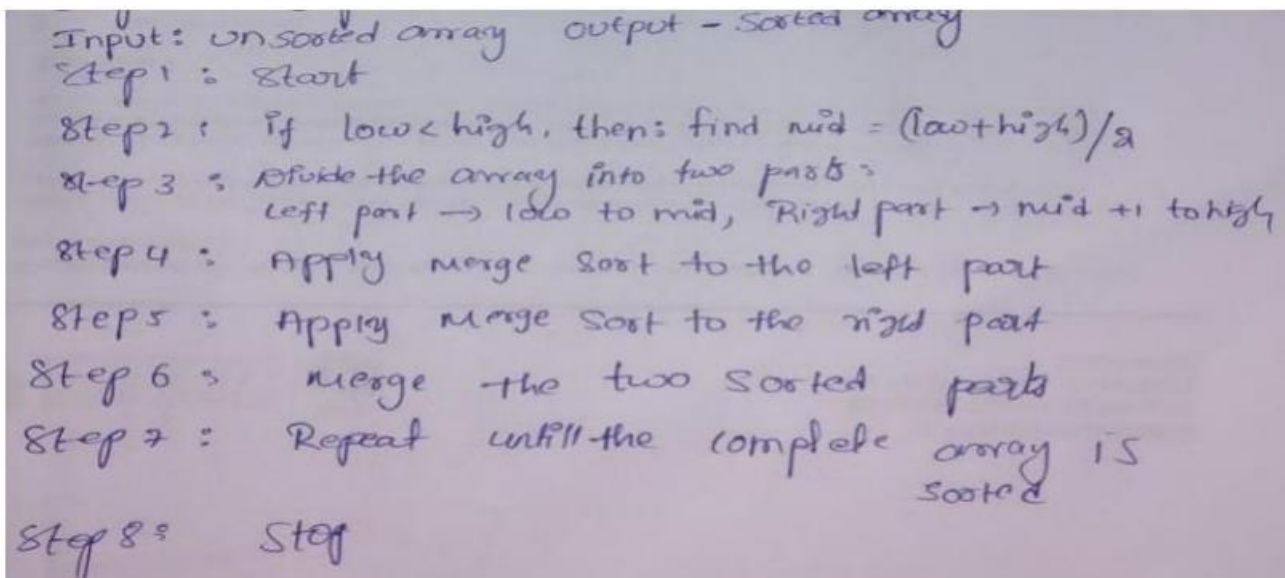
$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

MERGE SORT:

ALGORITHM:



Input: unsorted array output - sorted array
Step 1: Start
Step 2: If $low < high$, then: find $mid = (low + high) / 2$
Step 3: Divide the array into two parts:
Left part $\rightarrow low$ to mid , Right part $\rightarrow mid + 1$ to $high$
Step 4: Apply merge sort to the left part
Step 5: Apply merge sort to the right part
Step 6: Merge the two sorted parts
Step 7: Repeat until the complete array is sorted
Step 8: Stop

PROGRAM:

```
#include <iostream>

using namespace std;

class MergeSort
{
public:
    int size;
    int arr[100];

    void arrayInput()
    {
        cout << "Enter the size of array: ";
        cin >> size;

        cout << "Enter the array elements: ";
```



```
for (int i = 0; i < size; i++)  
{  
    cin >> arr[i];  
}  
}
```

```
void printArray()  
{  
    for (int i = 0; i < size; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}
```

```
void merge(int low, int mid, int high)  
{  
    int temp[100];  
    int i = low;  
    int j = mid + 1;  
    int k = low;  
  
    while (i <= mid && j <= high)  
    {  
        if (arr[i] <= arr[j])  
        {  
            temp[k] = arr[i];  
            i++;  
        }
```



```
    }  
    else  
    {  
        temp[k] = arr[j];  
        j++;  
    }  
    k++;  
}  
  
while (i <= mid)  
{  
    temp[k] = arr[i];  
    i++;  
    k++;  
}  
  
while (j <= high)  
{  
    temp[k] = arr[j];  
    j++;  
    k++;  
}  
  
for (i = low; i <= high; i++)  
{  
    arr[i] = temp[i];  
}  
}
```

```
void mergeLogic(int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        mergeLogic(low, mid);
        mergeLogic(mid + 1, high);
        merge(low, mid, high);
    }
}
};
```

```
int main()
{
    MergeSort ms;

    ms.arrayInput();

    cout << "Before sorting: ";
    ms.printArray();

    ms.mergeLogic(0, ms.size - 1);

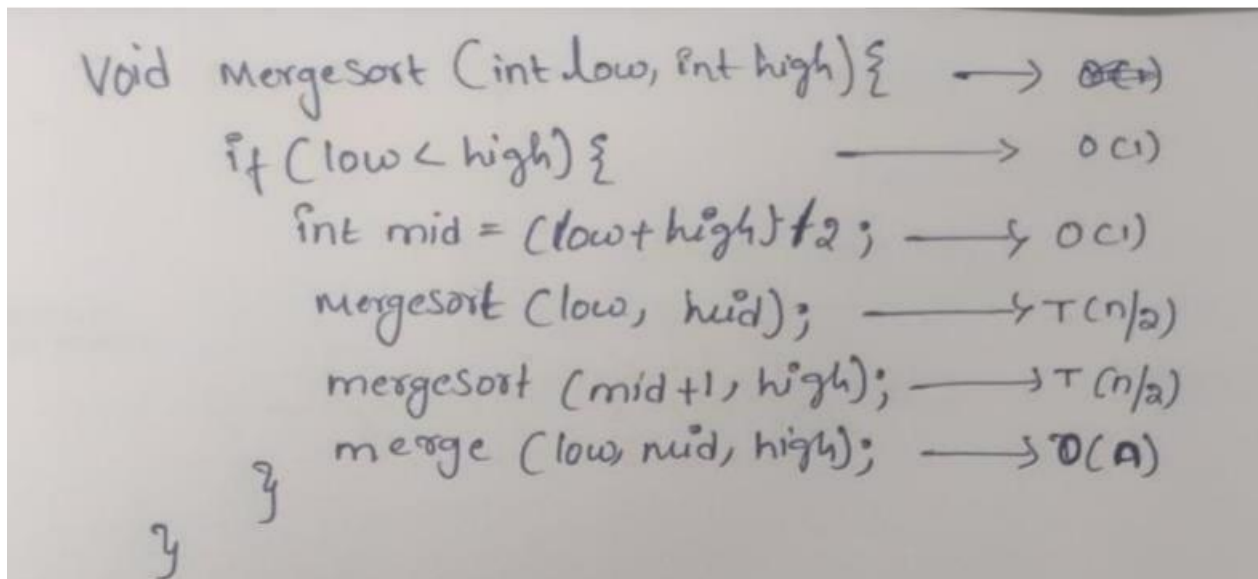
    cout << "After sorting: ";
    ms.printArray();
}
```

```
return 0;  
}
```

OUTPUT:

```
enter the size of array  
4  
enter the array elements  
2  
1  
4  
3  
before sorting  
2 1 4 3  
after sorting  
1 2 3 4  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:



```
Void mergesort (int low, int high) {  $\rightarrow O(n)$   
    if (low < high) {  $\rightarrow O(1)$   
        int mid = (low + high) / 2;  $\rightarrow O(1)$   
        mergesort (low, mid);  $\rightarrow T(n/2)$   
        mergesort (mid + 1, high);  $\rightarrow T(n/2)$   
        merge (low, mid, high);  $\rightarrow O(n)$   
    }  
}
```



Best case Best case is when merge sort performs the minimum amount of work.

$$T(n) = 2T(n/2) + O(n)$$

By using master method

general eq $T(n) = 2T(n/2) + f(n)$

$$\therefore a=2, b=2, k=1, p=0$$

$$\log_a b = \log_2 2 = 1$$

$$= 1$$

then $p > -1$ (case-2)

$$T(n) = \Theta(n^k (\log^{p+1} n))$$

$$T(n) = \Theta(n^1 (\log n))$$

$$T(n) = O(n \log n)$$

Average case occurs when array randomly arranged

- Divide the array into two halves
- Recursively sort both halves

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = O(n \log n)$$

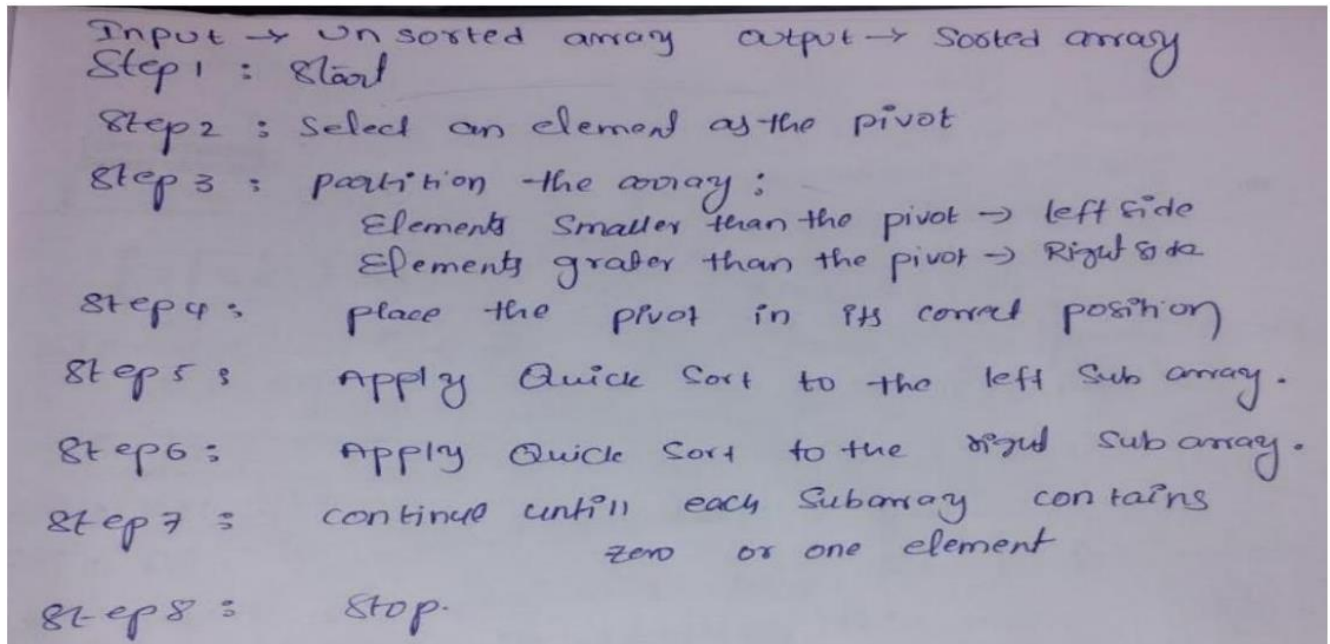
Worst case The input array may contain the maximum work during merging. Even in the worst case, merge sort divides the array into two equal halves.

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = O(n \log n)$$

QUICK SORT:

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

class QuickSort
{
public:
    int size;
    int arr[100];
    int i;

    void arrayInput()
    {
        cout << "Enter the size of array" << endl;
        cin >> size;
```

```
cout << "Enter the array elements" << endl;
for (i = 0; i < size; i++)
{
    cin >> arr[i];
}
}
```

```
void printArray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
int partition(int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    int temp;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
        }
    }
}
```



```
        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

```
temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;
```

```
return i + 1;
}
```

```
void quickLogic(int low, int high)
{
    if (low < high)
    {
        int pi = partition(low, high);

        quickLogic(low, pi - 1);
        quickLogic(pi + 1, high);
    }
}

};
```

```
int main()
{
```



```
QuickSort qs;
```

```
qs.arrayInput();
```

```
cout << "Before sorting" << endl;
```

```
qs.printArray();
```

```
qs.quickLogic(0, qs.size - 1);
```

```
cout << "After sorting" << endl;
```

```
qs.printArray();
```

```
return 0;
```

```
}
```

OUTPUT:

```
enter the size of array
3
enter the array elements
77
11
2
before sorting
77 11 2
after sorting
2 11 77

=== Code Execution Successful ===
```



TIME COMPLEXITY:

Handwritten code for quicksort with time complexity annotations:

```
void quicklogic (int low, int high) {  
    if (low < high) {  $\longrightarrow O(1)$   
        int pi = partition (low, high);  $\longrightarrow O(n)$   
        quicklogic (low, pi-1);  $\longrightarrow T(n/2)$   
        quicklogic (pi+1, high);  $\longrightarrow T(n/2)$   
    }  
}
```



Best case When the pivot divides the array into two approximately equal parts.

At entry level, partition takes $O(n)$

and array divide into 2 equal parts

$$T(n) = 2T(n/2) + O(n)$$

Master method

$$T(n) = aT(n/b) + O(n^p)$$

$a=2, b=2, f(n)=O(n)$

Now $\log_a b = \log_2 2 = 1$

Now case - 2

$$T(n) = O(n^k \times \log^{p+1} n)$$

$$T(n) = O(n \times \log n)$$

$$\boxed{T(n) = n \log n}$$

Average case

Occurs when the pivot partitions into reasonably balanced portions on average.

The recurrence is approximately:

$$T(n) = 2T(n/2) + O(n)$$

$a=2, b=2, f(n)=O(n)$

$$\log_a b = \log_2 2 = 1$$

$$T(n) = O(n^k \times \log^{p+1} n)$$

$$T(n) = O(n \times \log n)$$

$$\boxed{T(n) = O(n \log n)}$$

Worst case occurs when the selected pivot becomes the smallest or largest element every time. so only one element is removed at each partition.

The recurrence becomes:

$$T(n) = T(n-1) + O(n)$$

$$T(n) = T(n-1) + (n-1) \cdot n$$

$$T(n) = T(n-2) + T(n-1) + n$$

Continue

$$T(n) = 1 + 2 + 3 + \dots + n$$

$$T(n) = \frac{n(n-1)}{2} \quad T(n) = \frac{n^2 - n}{2} \quad \boxed{T(n) = O(n^2)}$$

CONCLUSION:

I understood that different sorting algorithms use different ways to arrange data. Bubble, Selection and Insertion Sort are easy to understand and implement, but they can take more time for large data. Merge Sort and Quick Sort use a divide-and-conquer approach and are generally more efficient for large datasets. Overall, choosing an algorithm depends on the size and condition of the data and the required